

DOI:10.3969/j.issn.1671-0673.2023.03.010

一种基于 API 执行序列的恶意代码检测方法

桂海仁^{1,2}, 刘福东^{1,2}, 王磊³, 牛超⁴

(1. 信息工程大学, 河南 郑州 450001; 2. 数学工程与先进计算国家重点实验室, 河南 郑州 450001;
3. 江西省军区数据信息室, 江西 南昌 330006; 4. 32090 部队, 河北 秦皇岛 066000)

摘要:随着软件开发技术的不断进步, 恶意代码愈发呈现功能复杂、隐蔽性强等特点。大多恶意代码具有一定的抗检测能力, 实际任务中容易导致漏报。针对恶意代码行为特点, 提出一种基于应用程序接口 (Application Programming Interface, API) 执行序列的恶意代码检测方法。首先利用 FastText 模型和平滑逆词频算法实现 API 序列的向量化表示, 再通过 TextCNN 卷积神经网络执行恶意代码检测。经实验测试表明, 所提方法在基准数据集上能有效提高恶意代码检测的准确率。

关键词:恶意代码检测; API 执行序列; 向量化; 卷积神经网络

中图分类号:TP309.5

文献标识码:A

文章编号:1671-0673(2023)03-0316-05

API Execution Sequences-Based Malware Detection Method

GUI Hairen^{1,2}, LIU Fudong^{1,2}, WANG Lei³, NIU Chao⁴

(1. Information Engineering University, Zhengzhou 450001, China; 2. State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou 450001, China; 3. Data Information Office of Jiangxi Provincial Military Region, Nanchang 330006, China; 4. Unit 32090, Qinhuangdao 066000, China)

Abstract: As software development technology progresses continuously, malwares are increasingly complex and concealed. Most malwares have certain anti-detection abilities, leading to false negatives in detection tasks. According to the behavior characteristics of malwares, a malware detection method based on the API execution sequences is proposed. The method adopts FastText model and Smooth Inverse Frequency algorithm to accomplish the vectorization representation of API sequences. Then malware detection is accomplished by convolutional neural network TextCNN. Experimental tests illustrate that the proposed method can effectively improve the accuracy of malware detection on the benchmark dataset.

Key words: malware detection; API execution sequences; vectorization; convolutional neural network

随着网络技术的发展, 恶意代码已成为网络空间安全的重要威胁。恶意代码检测是分析恶意代码的基础, 也是网络安全首当其冲的问题。为阻止恶意入侵及网络攻击, 需要提前检测出存在恶意行为的软件。动态检测方式虽然精确度较高, 但是执

行路径单一, 容易产生漏报问题。同时, 大多恶意软件都具有一定的抗检测能力, 发现虚拟环境后会执行规避操作, 导致检测失败。因此, 多数恶意代码方面的研究仍然围绕静态检测方法开展。

恶意代码功能复杂, 隐蔽性强, 但需要通过

收稿日期: 2022-04-06; 修回日期: 2022-04-25

基金项目: 国家自然科学基金资助项目 (61802435)

作者简介: 桂海仁 (1985-), 男, 博士生, 主要研究方向为网络安全、自然语言处理。

API 来访问并调用操作系统的底层服务。API 为二进制程序提供了函数封装,几乎所有的二进制程序都会使用操作系统提供的 API 接口来进行通信服务。因此,API 序列可以用来揭示恶意程序的行为信息,并用于恶意代码检测与分类。

由于恶意软件往往会产生家族变种样本,特征工程是一项重要而艰巨的工作。基于传统的方法,无法保证现有样本集的特征可用于未来的新型变种。深度学习有助于实现自动化特征学习,因此本文将 API 序列信息与神经网络模型结合,开展恶意代码检测任务。

1 研究现状

恶意代码检测是指通过软件功能和行为鉴别判定程序的良恶性^[1]。当前对恶意代码进行检测的方法主要分为3类:基于特征码的检测、启发式检测和基于特征学习的检测。

1.1 基于特征码检测

特征码扫描技术原理简单,扫描效率高,对用户计算机占用资源少。但这种方法只能扫描已知的样本。一个特征码是一个或一组简短的字节数组,对应一个已知的恶意样本,可准确检测出该样本并查杀。如遇新型的未知恶意代码或已知代码的新变种,扫描引擎必须提前获得该样本,并通过特征码生成算法,生成并存储唯一标识该样本的特征码。随着安全对抗技术的不断提高与增强,恶意组织也在不断开发和利用新的攻击手段和攻击载荷,仅仅依靠特征码技术准确检测恶意代码已不现实。

1.2 启发式检测

为了弥补特征码扫描技术在零日攻击样本的检测滞后问题,研究者提出了启发式检测技术。启发式的检测方法主要比较系统上层信息和系统内核,以鉴别新的可疑文件、注册表或进程操作。一段时期以来,许多研究都将启发式方法作为恶意代码检测的重点方向,并取得了一定的实际效果。启发式的检测方法严重依赖人工分析和先验知识,应用中容易产生误报和漏报;同时新型恶意代码样本不断出现,新式恶意攻击手段多样且隐蔽,传播速度迅猛,启发式检测技术也难以满足实际恶意代码检测需求。

1.3 基于特征学习的检测

近年来,深度学习技术不断发展,通过神经网络提取并选择样本关键特征的研究方式取得了广泛关注。不断有研究者将特征学习的方法引入恶

意代码检测。在恶意代码中代表恶意行为的特征分布通常是不均匀的,甚至会发生改变。同时由于训练集标签的误差普遍存在,难以保证数据集的标签准确性和样本的类别均匀。文献[2]将词频-逆向文件频率(Term Frequency-Inverse Document Frequency, TF-IDF)方法引入到恶意代码的特征选择上来,通过特征出现频率的统计并查找恶意代码可疑行为。文献[3]首次提出从二进制代码片段中提取恶意代码特征,并针对这些特征建立深度网络以分类恶意代码。文献[4]利用可执行文件的 API 调用序列动态分析数据集,基于 RNN 模型及其组合构建深度神经网络并进行对比测试。

本文提出了一种以 API 执行序列为特征的恶意代码检测方法,主要创新点包括:(1)基于控制流虚拟节点深度提取算法从恶意样本提取 API 执行序列;(2)提出一种信息增强的 API 序列恶意代码向量化模型;(3)基于 TextCNN 实现了恶意代码检测,测试得到的准确率指标高达 98.44%。

2 基于 API 序列的恶意代码检测方法

为更好地解决恶意代码检测过程中存在的问题,本文提出一种以 API 序列为特征的恶意代码检测方法,主要分为3个步骤:恶意代码的 API 执行序列提取、基于 API 执行序列的恶意代码向量化表示和基于 TextCNN 的恶意代码检测。下面分别详细介绍。

2.1 恶意代码 API 执行序列提取

包括恶意程序在内的绝大多数二进制程序均通过操作系统提供的 API 接口来调用底层服务与功能。例如,恶意软件通常会创建或打开现有文件,该程序将调用 OpenFileW 系统 API。DLL 注入技术作为恶意软件中常见的恶意行为,其往往与 WriteProcessMemory、LoadLibrary 和 CreateRemoteThread 等 API 息息相关。已有相关研究表明,通过对恶意代码进行分析,部分 API 行为为恶意样本经常使用,如 GetTickCount、GetModuleHandleW 等。根据以上判断,API 序列可用于揭示恶意程序的行为信息,并用于恶意代码检测。

为生成恶意代码执行序列的表征向量,主要采用静态方式分析可执行文件。使用 IDA 反编译工具对恶意样本进行反汇编,得到包含地址、指令、注释和 API 执行等信息的 idb 文件。再通过 IDAPython 插件,实现恶意代码 API 执行序列的控制流

虚拟节点深度提取算法。算法描述如下:(1)依据 IDA 提供的接口对汇编指令进行处理,得到基本块的地址、基本块间的跳转关系以及 API 函数调用地址,构建以基本块为单位的控制流图;(2)对各节点中的 API 函数执行序列进行搜索并提取生成虚拟节点,节点中仅包含 API 执行信息;(3)消除空节点,即基本块中无相关 API 执行的节点,获取 API_SEQ_CFG 图;(4)依据深度优先算法对 API_SEQ_CFG 进行 API 序列提取,得到全路径下的 API 序列。

2.2 基于 API 执行序列的恶意代码向量化

(1)恶意代码向量化模型总体概述。提出了一种基于 API 执行序列的恶意代码向量化模型,其流程如图 1 所示。

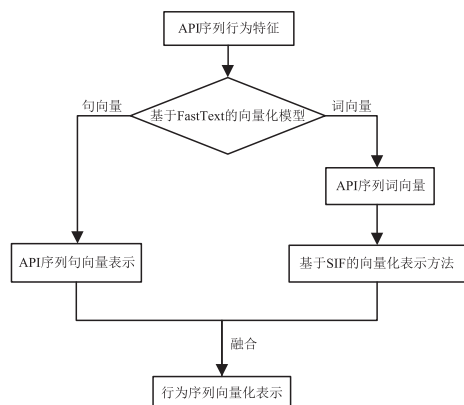


图 1 基于 API 序列的恶意代码向量化流程图

该模型中首先将 API 执行序列行为特征输入至基于 FastText 的向量化模型,该模型可以在词向量化表示的同时,实现对句子的向量化表示。计算句子中每个词向量的 $L2$ 范数,并用每个词向量除以其 $L2$ 范数,最后将词向量进行平均得到句子向量。由于恶意代码行为中,每个序列的行为都有不同的特征,如注册表和网络行为。因此,通过 FastText 对恶意代码的每个行为进行向量化,也可以实现对恶意样本的向量化表示。

使用 FastText 得到 API 序列词向量,进一步增强 API 序列向量的语义。在此基础上,基于平滑逆词频(Smooth Inverse Frequency, SIF)^[5]方法,从加权平均的角度上实现了行为序列向量化表示。

最终,对两种行为序列向量进行特征融合,实现基于 API 序列的恶意样本向量化表示。在具体实现中,主要采用拼接方式对二者进行特征融合。

(2)基于 FastText 的向量化模型。FastText^[6]是一种文本表示方法,其原理与 Word2Vec 的连词袋模型(Continuous Bag-of-Words Model, CBOW)

相似。该方法提出基于 n -gram 进行句子学习表示,是 word2vec 模型中 CBOW 形式的扩展。FastText 使用所有的单词和相应的 n -gram 特征作为输入,并学习句子的向量表示,用于预测句子类别。

FastText 模型有 3 层:输入层、隐藏层和输出层,其体系结构与 CBOW 相似。FastText 的输入是多个单词及其 n -gram 特征,输出是一个特定的目标,隐含层是多个单词向量的叠加平均。CBOW 的输出是目标词汇表,而 FastText 的输出是样本的相应类别。FastText 模型结构如图 2 所示。

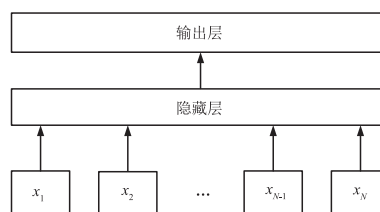


图 2 FastText 模型示意图

FastText 的输入为 $\{x_1, x_2, \dots, x_N\}$, 即 N 个 n -gram 特征。对于进程行为 API “TerminateProcess”, 长度为 5 的字符级别 n -gram 则为 “Termi”、“ermin” 和 “rmina” 等。

隐藏层输出计算方法表示为

$$h = \frac{1}{N} \left(\sum_{i=1}^N x_i W \right) \quad (1)$$

式中 W 表示输入到隐藏层的权重。输出层计算方法表示为

$$y = \text{softmax}(W' \cdot \text{sigmoid}(h)) \quad (2)$$

式中 W' 表示隐藏层到输出层的权重。

(3)基于 SIF 的向量化表示方法。恶意代码的 API 序列揭示恶意代码的行为,其调用频次通常具有强烈的指示信息。例如,在恶意样本中,进程监控相关的 API 将被多次调用。因此,对于恶意代码分析而言,对不同 API 进行权重加权得到的序列向量更为合理。综上考虑,可以选择基于组成词向量的文本嵌入方法 SIF 生成 API 序列向量。

SIF 向量化的具体过程为:首先计算句子中组成词向量的加权平均;然后计算完所有的均值后, SIF 计算第一个主成分;最后,去除所有的句子向量在第一个主成分上的投影。依据文献[7],这种常见成分的去除减少了句子嵌入所包含的句法信息的权重,从而使语义信息在向量表示中权值更大。

从恶意行为加权的角度,基于 SIF 模型计算词嵌入向量的加权平均,作为每个恶意样本执行序列的初始嵌入向量。并用主成分分析法对这些初始的执行序列嵌入向量进行修正,得到每个恶意样本

的SIF向量化表示。最终,可通过拼接方式将FastText向量与SIF向量进行特征融合,实现基于API序列的恶意样本向量化表示。

2.3 基于TextCNN的恶意代码检测

利用上述提出的恶意样本向量,基于TextCNN构建恶意代码检测模型。TextCNN模型是CNN模型的一个变体。它能充分发挥CNN的并行计算能力,训练速度更快。除了保留原始CNN的特征外,它还增加了提取文本特征的能力。TextCNN使用一维卷积来获取句子的n-gram特征表示,具有很强的浅层文本特征提取能力。

TextCNN模型如图3所示,包括嵌入层、卷积层、池化层、全连接层和Softmax层。在具体实现中,将原始结构中的嵌入(embedding)层替换为API执行序列向量化模型,并将恶意样本中各函数按照函数地址进行排序,从而实现将恶意样本的语义信息完整地输入至TextCNN模型。

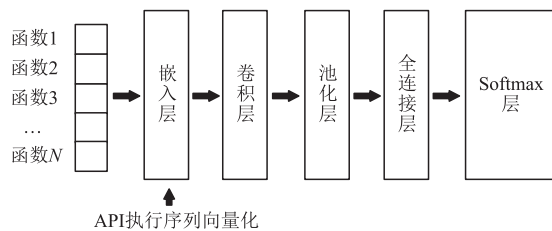


图3 TextCNN模型结构图

在提出的向量化模型的基础上,利用卷积层对其进行卷积操作,并选择Relu作为激活函数获取特征映射图(feature map)。模型所采用的3种类型的卷积窗口,并分别设置窗口大小为 $2 * 256$ 、 $4 * 256$ 和 $8 * 256$ 。其中256为通过FastText嵌入向量与SIF计算向量级联后的向量维度。

接下来,利用池化层对各特征映射图进行池化操作。具体采用1-max池化层,即提取每个特征映射图向量中的最大值,该向量可以捕获其最重要的特征。最后,基于全连接层和Softmax层对结果进行分类输出,实现恶意样本的恶意性检测。

3 实验与分析

为验证提出的基于API执行序列的恶意代码检测方法的有效性,进一步开展了相关实验和测试。

3.1 数据集与实验设置

本实验以广泛应用于恶意代码分析任务的VXHeaven^[8]作为数据集,该数据集提供4个字段的标签,按顺序依次为病毒种类、所属平台、家族、变种信息。本实验分类随机抽取50 000个样本作

为恶意样本。再从windows系统文件和C/C++开源项目中随机抽取50 000个程序作为良性样本,以此构建含有100 000个样本的数据集。

实验平台CPU为Intel Core i7-8700 K,显卡为GeForce 1050 Ti,内存大小为32 G,磁盘为4 T容量的固态硬盘。

二进制文件通过IDA反汇编后,输入至API序列向量化模型进行训练,从而对可执行程序进行向量化。将该向量输入至TextCNN模型中训练并进行恶意代码检测实验。

3.2 恶意代码API向量可视化

API序列向量化模型中,依据模型参数设置,可将API表示为128维的向量。在可视化实验中,首先基于非线性降维(T-distributed Stochastic Neighbor Nembedding, T-SNE)算法将1 383个API向量进行降维可视化,结果如图4所示。

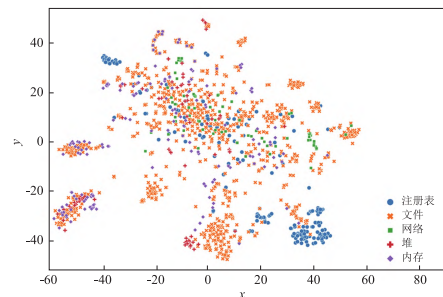


图4 基于T-SNE的部分API向量可视化结果

依据提出的划分方式,涉及向量涵盖注册表、文件、网络、堆和内存操作相关API。不同颜色的点代表不同种类的API,颜色不同的点能够按照一定的聚合规律在范围内聚集,注册表行为相关的API较为集中,并与其他类别区分较为明显。同时,由于各类别数量分布不均衡,文件API数量明显更多。由可视化结果可知,文件API都相对集中在各个区域,并且在部分簇中,文件、内存与堆之间的API距离较为接近。这是因为文件操作往往与内存和堆之间紧密相关,并且内存操作和堆操作语义接近,这在一定程度上证明了API序列向量化的科学性。

为充分阐述API向量化的合理性,本实验基于主成分分析(Principal Component Analysis, PCA)算法对API向量进行降维及可视化。由于样本总数不均衡,为方便展示,实验从5种API类别中分别随机选择5个API进行可视化实验。实验结果如图5所示,API序列向量化方法可学习对应的语义信息,并用于恶意样本检测。因此,序列向量化方法能有效提取程序中不同API的类别信息。

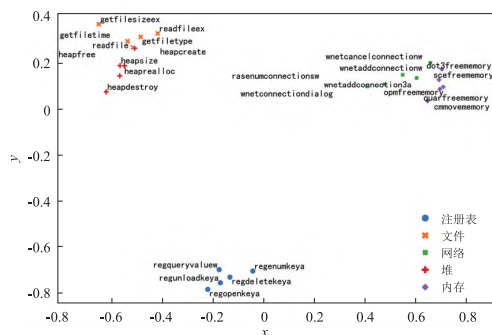


图5 基于PCA的部分API向量可视化结果

3.3 恶意代码检测对比结果

为评估本文提出的API序列向量化方法对恶意代码检测的有效性,实验使用TextCNN作为恶意代码检测模型,并与其他分类模型进行对比。本实验将LSTM、BiLSTM、GRU、BiGRU及CNN作为模型参照,以评估基于API执行序列的恶意代码检测模型的性能,具体结果如表1所示。

表1 不同模型的恶意代码检测结果对比表

模型	准确率	精确率	F1值
LSTM	0.970 2	0.974 3	0.964 7
BiLSTM	0.978 3	0.980 5	0.971 1
GRU	0.972 6	0.970 2	0.968 5
BiGRU	0.979 5	0.974 5	0.972 0
CNN	0.968 2	0.958 1	0.960 4
TextCNN	0.984 4	0.986 2	0.980 7

从表1可知,TextCNN的性能在准确率、精确率及F1值多个实验指标中都优于其他常见的网络结构。其中,TextCNN的准确率达到98.44%,高于CNN(96.82%)。TextCNN比常见的神经网络方法在恶意代码检测上具有更好的性能。与此同时,API序列向量化方法无论在常见的神经网络结构还是TextCNN上都具备恶意代码检测能力,证实了所提方法的有效性。

本实验在训练过程中,epoch设置为10,具体训练结果如图6所示。在训练到第8轮时,每个模型基本都已达到稳定的结果。整体来看,相比其他模型而言,TextCNN训练速度最快并且准确率最高。

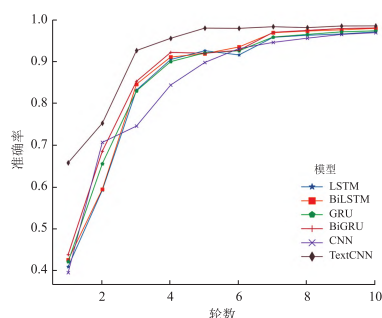


图6 恶意代码检测准确率训练结果比较曲线图

4 结束语

本文通过构建一种以API执行序列为研究对象的恶意代码检测框架,该框架利用FastText模型和SIF算法对API序列实现二进制函数级的向量化表示,并通过TextCNN进行恶意代码检测应用。经过实验验证与对比,该向量化方法能有效提取二进制代码的功能特征,检测结果相比常见模型能有效提高准确性指标,证实了提出的API执行序列向量化表示方法在恶意代码检测任务中的有效性。

参考文献:

- [1] 王蕊,冯登国,杨轶,等.基于语义的恶意代码行为特征提取及检测方法[J].软件学报,2012,23(2):378-393.
- [2] YUAN H L, TANG Y C, SUN W J, et al. A detection method for android application security based on TF-IDF and machine learning [J]. PLoS One, 2020, 15(9):e0238694.
- [3] SAXE J, BERLIN K. Deep neural network based malware detection using two dimensional binary program features [C]//2015 10th International Conference on Malicious and Unwanted Software (MALWARE). Fajardo, PR, USA: IEEE, 2015: 11-20.
- [4] ATHIWARATKUN B, STOKES J W. Malware classification with LSTM and GRU language models and a character-level CNN [C]//2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). New Orleans, LA, USA: IEEE, 2017: 2482-2486.
- [5] ARORA S, LIANG Y Y, MA T Y. A simple but tough-to-beat baseline for sentence embeddings [C]//5th International Conference on Learning Representations. Toulon, France: Openreview, 2017: 1-16.
- [6] BOJANOWSKI P, GRAVE E, JOULIN A, et al. Enriching word vectors with subword information [J]. Transactions of the Association for Computational Linguistics, 2017, 5: 135-146.
- [7] ETHAYARAJH K. Unsupervised random walk sentence embeddings: a strong but simple baseline [C]//Proceedings of The Third Workshop on Representation Learning for NLP. Melbourne, Australia: Association for Computational Linguistics, 2018: 91-100.
- [8] VX Heaven. VX Heaven virus collection 2010-05-18 [DS/OL]. (2010-05-18) [2022-03-26]. <https://academictorrents.com/details/34ebe49a48aa532deb9c0dd08a08a017aa04d810>.

(编辑:高明霞)